

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250410>

Outline : Reinforcement learning {TF support} cont. (code [samples](#))

- Definitions / Motivation
 - Deep Q-Learning Variants
 - The TF-Agents Library
 - Overview of Some Popular RL Algorithms
 - Takeaways
-

Source: Aurelien Geron, “Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems”, 2019 ([Available online](#)) / **Chapter 18**

Definition / Motivation (1)

Playing Atari with Deep Reinforcement Learning (Videos)

Human-level control through deep reinforcement learning

In Reinforcement Learning, a software *agent* makes observations and takes *actions* within an *environment*, and in return it receives *rewards*. Its objective is to learn to act in a way that will maximize its expected rewards over time.

Definition / Motivation (2)

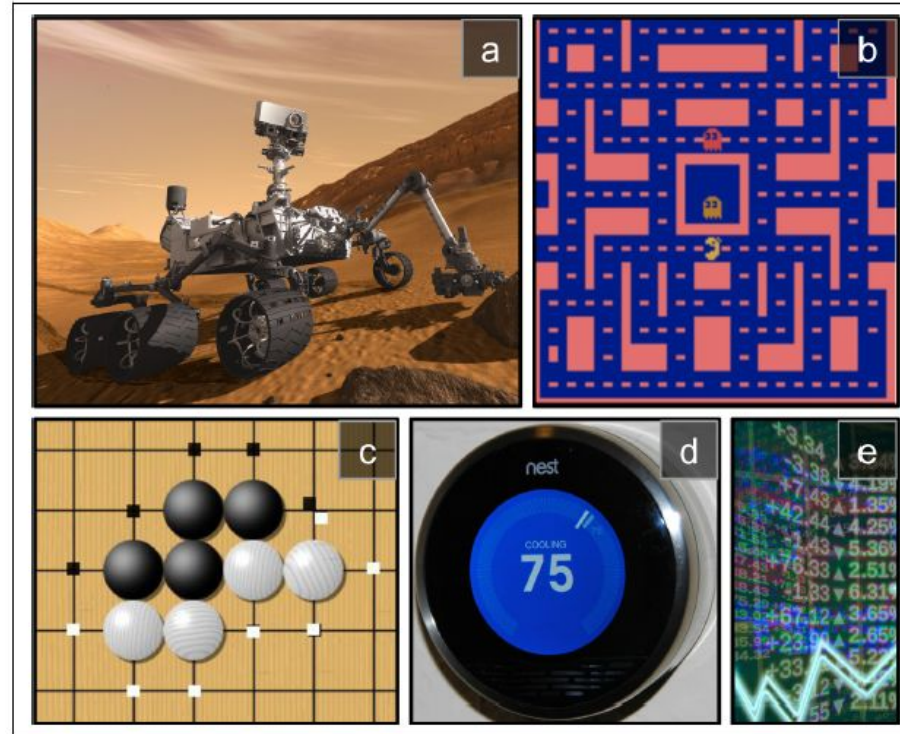


Figure 18-1. Reinforcement Learning examples: (a) robotics, (b) Ms. Pac-Man, (c) Go player, (d) thermostat, (e) automatic trader⁵

Markov Decision Processes (1)

- *Markov chains*: stochastic processes with no memory.
 - Fixed number of states, and it randomly evolves from one state to another at each step.
 - The probability for it to evolve from a state s to a state s' is fixed, and it depends only on the pair (s, s') , not on past states (so, the system has no memory).

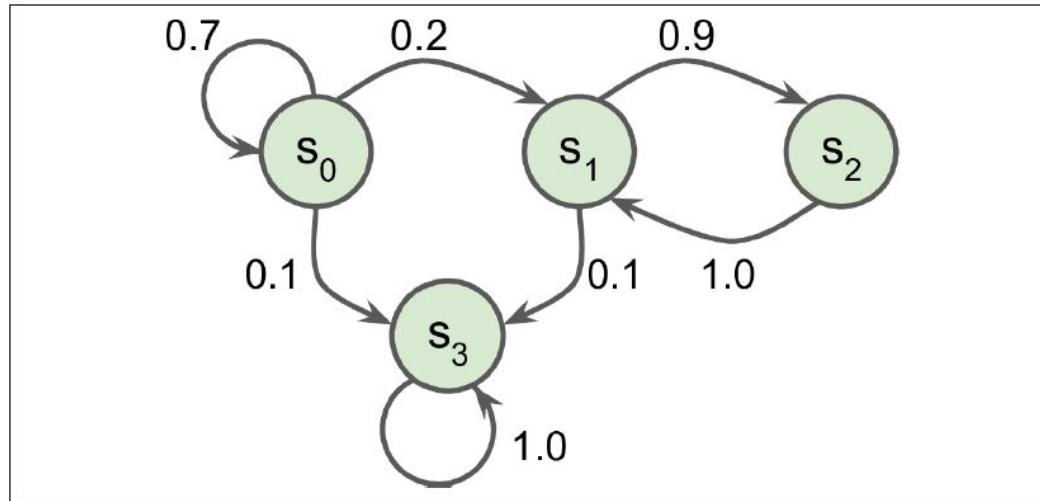


Figure 18-7. Example of a Markov chain

Markov Decision Processes (2)

- **Markov decision processes** (MDP) were first described in the 1950s.
- MDP resemble Markov chains but with a twist:
 - at each step, an agent can **choose one of several possible actions**, and
 - the **transition probabilities depend on the chosen action**.
 - **some state transitions return some reward** (positive or negative), and
 - the agent's goal is to find a policy that will **maximize reward over time**.

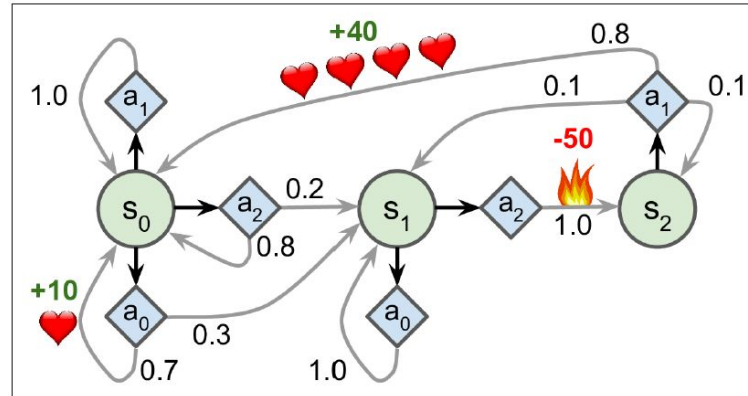


Figure 18-8. Example of a Markov decision process

Markov Decision Processes : Bellman Optimality Equation

Equation 18-1. Bellman Optimality Equation

$$V^*(s) = \max_a \sum_s T(s, a, s') [R(s, a, s') + \gamma \cdot V^*(s')] \quad \text{for all } s$$

- $T(s, a, s')$ is the transition probability from state s to state s' , given that the agent chose action a . For example, in Figure, $T(s_2, a_1, s_0) = 0.8$.
- $R(s, a, s')$ is the reward that the agent gets when it goes from state s to state s' , given that the agent chose action a . For example, in Figure, $R(s_2, a_1, s_0) = +40$.
- γ is the discount factor.

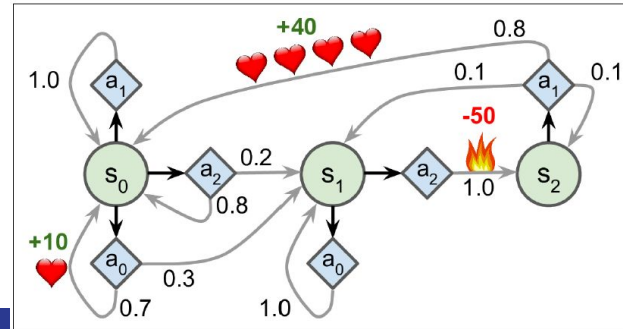


Figure 18-8. Example of a Markov decision process

Deep Q-Learning Variants

Fixed Q-Value Targets

- Basic Deep Q-Learning algorithm, the model is used both to make prediction and to set its own targets.
- This feedback loop can make the network unstable: it can diverge, oscillate, freeze...
- Two DQNs instead of one: the first is the online model, which learns at each step and is used to move the agent around, and the other is the target model used only to define the targets.

- The target model is just a clone of the online model:

```
target = keras.models.clone_model(model)
target.set_weights(model.get_weights())
```

- copying the weights of the online model to the target model, at regular intervals (e.g., every 50 episodes):

```
if episode % 50 == 0:
    target.set_weights(model.get_weights())
```

Double DQN

- Double DQN. The update was based on the observation that the target network is prone to overestimating Q-Values.
- Suppose all actions are equally good: the Q-Values estimated by the target model should be identical, but since they are approximations, some may be slightly greater than others, by pure chance.
- Using the online model instead of the target model when selecting the best actions for the next states, and using the target model only to estimate the Q-Values for these best actions.

```
def training_step(batch_size):
    experiences = sample_experiences(batch_size)
    states, actions, rewards, next_states, dones = experiences
    next_Q_values = model.predict(next_states)
    best_next_actions = np.argmax(next_Q_values, axis=1)
    next_mask = tf.one_hot(best_next_actions, n_outputs).numpy()
    next_best_Q_values = (target.predict(next_states) * next_mask).sum(axis=1)
    target_Q_values = (rewards +
                      (1 - dones) * discount_factor * next_best_Q_values)
    mask = tf.one_hot(actions, n_outputs)
    [...] # the rest is the same as earlier
```

Prioritized Experience Replay

- *Importance sampling* (IS) or prioritized experience replay (PER): sample important experiences more frequently than uniformly from the replay buffer.
- Experiences are considered “important” if they are likely to lead to fast learning progress.
 - So, measure the magnitude of the TD error: $\delta = r + \gamma \cdot V(s') - V(s)$.
 - A large TD error indicates that a transition (s, r, s') is very surprising, and thus probably worth learning from (could also just be that the rewards are noisy).
 - When an experience is recorded in the replay buffer, its priority is set to a very large value, to ensure that it gets sampled at least once. However, once it is sampled (and every time it is sampled), the TD error δ is computed, and this experience's priority is set to $p = |\delta|$ (plus a small constant to ensure that every experience has a nonzero probability of being sampled).

Dueling DQN

- The Dueling DQN algorithm: Q-Value of a state-action pair (s, a) can be expressed as $Q(s, a) = V(s) + A(s, a)$; *value* of a state + *advantage* of taking action a in state s .
- Value of a state is equal to the Q-Value of the best action a^* for that state (since we assume the optimal policy will pick the best action), so $V(s) = Q(s, a^*)$, thus $A(s, a^*) = 0$.
 - The model estimates both the value of the state and the advantage of each possible action. Since the best action should have an advantage of 0, the model subtracts the maximum predicted advantage from all predicted advantages.

```
K = keras.backend
input_states = keras.layers.Input(shape=[4])
hidden1 = keras.layers.Dense(32, activation="elu")(input_states)
hidden2 = keras.layers.Dense(32, activation="elu")(hidden1)
state_values = keras.layers.Dense(1)(hidden2)
raw_advantages = keras.layers.Dense(n_outputs)(hidden2)
advantages = raw_advantages - K.max(raw_advantages, axis=1, keepdims=True)
Q_values = state_values + advantages
model = keras.Model(inputs=[input_states], outputs=[Q_values])
```

Deep Q-Learning Variants : Summary

Implementing all of these techniques, debugging them, fine-tuning them, and of course training the models can require a huge amount of work.

So instead of reinventing the wheel, it is often best to reuse scalable and well-tested libraries, such as **TF-Agents**.

The TF-Agents Library

TF-Agents Library

- Reinforcement Learning library based on TensorFlow
- Provides many off-the-shelf environments (including wrappers for all OpenAI Gym / [Gymnasium](#) environments), plus it supports the [PyBullet](#) library (for 3D physics simulation), [DeepMind's DM Control library](#) (based on [MuJoCo's](#) physics engine), and [Unity's ML-Agents](#) library (simulating many 3D environments).
- Implements many RL algorithms, including REINFORCE, DQN, and DDQN, as well as various RL components such as efficient replay buffers and metrics.

TF-Agents Library : Breakout game



Figure 18-11. The famous Breakout game

Installing TF-Agents

```
$ python3 -m pip install -U tf-agents
```

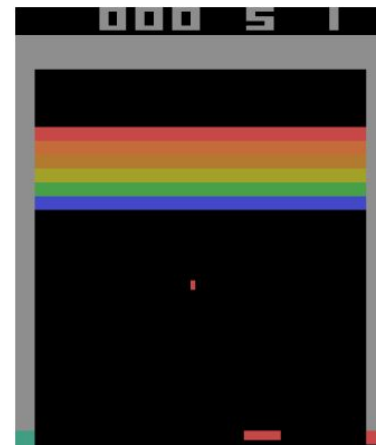
TF-Agents Environments

```
>>> from tf_agents.environments import suite_gym
>>> env = suite_gym.load("Breakout-v4")
>>> env
<tf_agents.environments.wrappers.TimeLimit at 0x10c523c18>
```

```
>>> env.gym
<gym.envs.atari.atari_env.AtariEnv at 0x24dcab940>
```

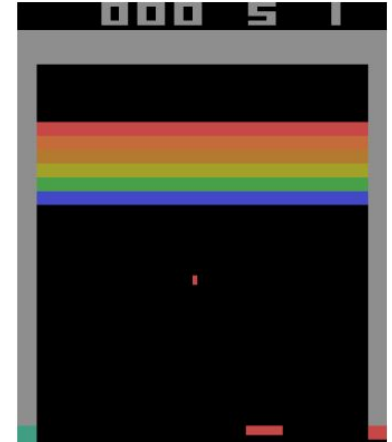
```
>>> env.reset()
TimeStep(step_type=array(0, dtype=int32),
          reward=array(0., dtype=float32),
          discount=array(1., dtype=float32),
          observation=array([[[0., 0., 0.], [0., 0., 0.],...]]], dtype=float32))
```

```
>>> env.step(1) # Fire
TimeStep(step_type=array(1, dtype=int32),
          reward=array(0., dtype=float32),
          discount=array(1., dtype=float32),
          observation=array([[[0., 0., 0.], [0., 0., 0.],...]]], dtype=float32))
```



Environment Specifications

```
>>> from tf_agents.environments import suite_gym
>>> env = suite_gym.load("Breakout-v4")
>>> env
<tf_agents.environments.wrappers.TimeLimit at 0x10c523c18>
```



```
>>> env.observation_spec()
BoundedArraySpec(shape=(210, 160, 3), dtype=dtype('float32'), name=None,
                  minimum=[[[0. 0. 0.], [0. 0. 0.], ...]],
                  maximum=[[[255., 255., 255.], [255., 255., 255.], ...]])
```

```
>>> env.action_spec()
BoundedArraySpec(shape=(), dtype=dtype('int64'), name=None,
                  minimum=0, maximum=3)
```

```
>>> env.time_step_spec()
TimeStep(step_type=ArraySpec(shape=(), dtype=dtype('int32'), name='step_type'),
         reward=ArraySpec(shape=(), dtype=dtype('float32'), name='reward'),
         discount=BoundedArraySpec(shape=(), ..., minimum=0.0, maximum=1.0),
         observation=BoundedArraySpec(shape=(210, 160, 3), ...))
```

```
>>> env.gym.get_action_meanings()
['NOOP', 'FIRE', 'RIGHT', 'LEFT']
```

Environment Wrappers and Atari Preprocessing (1)

Tf_agents.environments.wrappers

...

ActionClipWrapper

ActionDiscretizeWrapper

ActionRepeat

RunStats

TimeLimit

```
from tf_agents.environments.wrappers import ActionRepeat  
  
repeating_env = ActionRepeat(env, times=4)
```

Environment Wrappers and Atari Preprocessing (2)

Breakout environment that will run for a maximum of 10,000 steps during each episode, and each action will be repeated four times:

```
from gym.wrappers import TimeLimit

limited_repeating_env = suite_gym.load(
    "Breakout-v4",
    gym_env_wrappers=[lambda env: TimeLimit(env, max_episode_steps=10000)],
    env_wrappers=[lambda env: ActionRepeat(env, times=4)])
```

Environment Wrappers and Atari Preprocessing (3)

AtariPreprocessing wrapper - supported preprocessing steps:

Grayscale and downsampling: Observations are converted to grayscale and downsampled (by default to 84×84 pixels).

Max pooling: The last two frames of the game are max-pooled using a 1×1 filter. This is to remove the flickering that occurs in some Atari games due to the limited number of sprites that the Atari 2600 could display in each frame.

Frame skipping: The agent only gets to see every n frames of the game (by default $n = 4$), and its actions are repeated for each frame, collecting all the rewards. This effectively speeds up the game from the perspective of the agent, and it also speeds up training because rewards are less delayed.

End on life lost: In some games, the rewards are just based on the score, so the agent gets no immediate penalty for losing a life.

Environment Wrappers and Atari Preprocessing (4)

```
from tf_agents.environments import suite_atari
from tf_agents.environments.atari_preprocessing import AtariPreprocessing
from tf_agents.environments.atari_wrappers import FrameStack4

max_episode_steps = 27000 # <=> 108k ALE frames since 1 step = 4 frames
environment_name = "BreakoutNoFrameskip-v4"

env = suite_atari.load(
    environment_name,
    max_episode_steps=max_episode_steps,
    gym_env_wrappers=[AtariPreprocessing, FrameStack4])
```

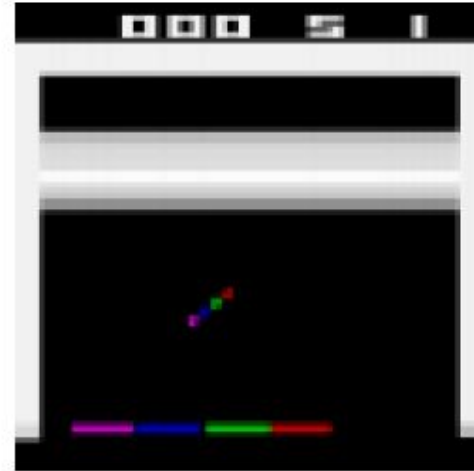


Figure 18-12. Preprocessed Breakout observation

Training Architecture (1)

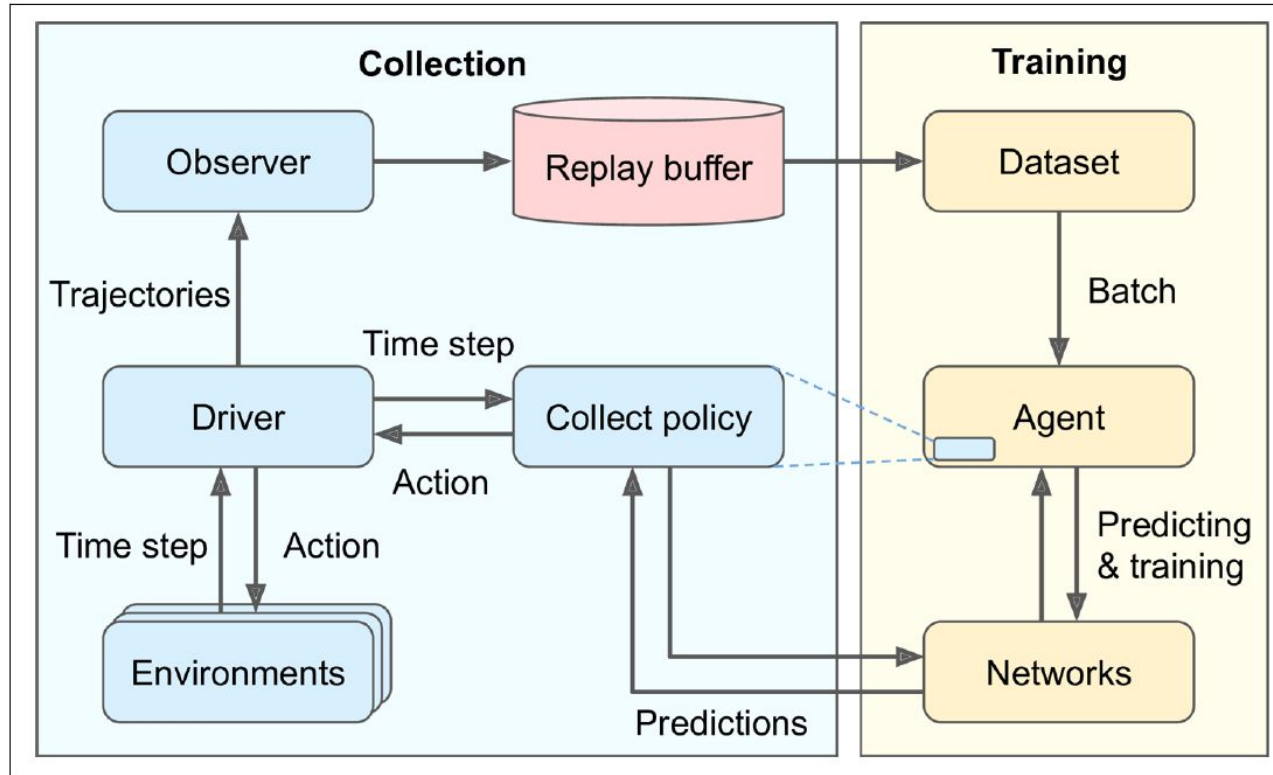


Figure 18-13. A typical TF-Agents training architecture

Training Architecture (2)

Left: a *driver* explores the *environment* using a *collect policy* to choose actions, and it collects *trajectories* (i.e., experiences), sending them to an *observer*, which saves them to a replay buffer.

Right: an *agent* pulls batches of trajectories from the replay buffer and trains some *networks*, which the collect policy uses.

So, the left part explores the environment and collects trajectories, while the right part learns and updates the collect policy.

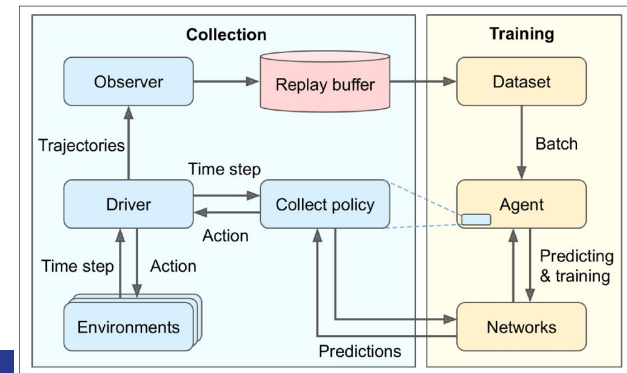


Figure 18-13. A typical TF-Agents training architecture

Creating the Deep Q-Network (1)

tf_agents.networks

```
from tf_agents.networks.q_network import QNetwork

preprocessing_layer = keras.layers.Lambda(
    lambda obs: tf.cast(obs, np.float32) / 255.)
conv_layer_params=[(32, (8, 8), 4), (64, (4, 4), 2), (64, (3, 3), 1)]
fc_layer_params=[512]

q_net = QNetwork(
    tf_env.observation_spec(),
    tf_env.action_spec(),
    preprocessing_layers=preprocessing_layer,
    conv_layer_params=conv_layer_params,
    fc_layer_params=fc_layer_params)
```

Creating the Deep Q-Network (2)

- Preprocessing layer: a simple Lambda layer that casts the observations to 32-bit floats and normalizes them (the values will range from 0.0 to 1.0).
- The observations contain unsigned bytes, which use 4 times less space than 32-bit floats, which is why we did not cast the observations to 32-bit floats earlier; we want to save RAM in the replay buffer.
- Next, the network applies three convolutional layers: the first has 32 8×8 filters and uses a stride of 4, the second has 64 4×4 filters and a stride of 2, and the third has 64 3×3 filters and a stride of 1. Lastly, it applies a dense layer with 512 units, followed by a dense output layer with 4 units, one per Q-Value to output (i.e., one per action).

Creating the DQN Agent

```
from tf_agents.agents.dqn.dqn_agent import DqnAgent

train_step = tf.Variable(0)
update_period = 4 # train the model every 4 steps
optimizer = keras.optimizers.RMSprop(lr=2.5e-4, rho=0.95, momentum=0.0,
                                     epsilon=0.00001, centered=True)
epsilon_fn = keras.optimizers.schedules.PolynomialDecay(
    initial_learning_rate=1.0, # initial  $\epsilon$ 
    decay_steps=250000 // update_period, #  $\Leftrightarrow$  1,000,000 ALE frames
    end_learning_rate=0.01) # final  $\epsilon$ 
agent = DqnAgent(tf_env.time_step_spec(),
                 tf_env.action_spec(),
                 q_network=q_net,
                 optimizer=optimizer,
                 target_update_period=2000, #  $\Leftrightarrow$  32,000 ALE frames
                 td_errors_loss_fn=keras.losses.Huber(reduction="none"),
                 gamma=0.99, # discount factor
                 train_step_counter=train_step,
                 epsilon_greedy=lambda: epsilon_fn(train_step))
agent.initialize()
```

Creating the Replay Buffer and the Corresponding Observer

Replay buffer with uniform sampling

```
from tf_agents.replay_buffers import tf_uniform_replay_buffer

replay_buffer = tf_uniform_replay_buffer.TFUniformReplayBuffer(
    data_spec=agent.collect_data_spec,
    batch_size=tf_env.batch_size,
    max_length=1000000)

class ShowProgress:
    def __init__(self, total):
        self.counter = 0
        self.total = total
    def __call__(self, trajectory):
        if not trajectory.is_boundary():
            self.counter += 1
        if self.counter % 100 == 0:
            print("\r{}/{}".format(self.counter, self.total), end="")
```

Creating Training Metrics

tf_agents.metrics

```
train_metrics = [  
    tf_metrics.NumberOfEpisodes(),  
    tf_metrics.EnvironmentSteps(),  
    tf_metrics.AverageReturnMetric(),  
    tf_metrics.AverageEpisodeLengthMetric(),  
]
```

```
>>> from tf_agents.eval.metric_utils import log_metrics  
>>> import logging  
>>> logging.getLogger().set_level(logging.INFO)  
>>> log_metrics(train_metrics)  
[...]  
NumberOfEpisodes = 0  
EnvironmentSteps = 0  
AverageReturn = 0.0  
AverageEpisodeLength = 0.0
```

Creating the Collect Driver

tf_agents.drivers

```
from tf_agents.drivers.dynamic_step_driver import DynamicStepDriver

collect_driver = DynamicStepDriver(
    tf_env,
    agent.collect_policy,
    observers=[replay_buffer_observer] + training_metrics,
    num_steps=update_period) # collect 4 steps for each training iteration

from tf_agents.policies.random_tf_policy import RandomTFPolicy

initial_collect_policy = RandomTFPolicy(tf_env.time_step_spec(),
                                         tf_env.action_spec())

init_driver = DynamicStepDriver(
    tf_env,
    initial_collect_policy,
    observers=[replay_buffer.add_batch, ShowProgress(20000)],
    num_steps=20000) # <=> 80,000 ALE frames
final_time_step, final_policy_state = init_driver.run()
```


Creating the Dataset

Sample a small batch of **two trajectories (subepisodes)**, each containing **three consecutive steps**. These subepisodes are shown in Figure 18-15 (each row contains three consecutive steps from an episode):

```
>>> trajectories, buffer_info = replay_buffer.get_next(
...     sample_batch_size=2, num_steps=3)
...
>>> trajectories._fields
('step_type', 'observation', 'action', 'policy_info',
 'next_step_type', 'reward', 'discount')
>>> trajectories.observation.shape
TensorShape([2, 3, 84, 84, 4])
>>> trajectories.step_type.numpy()
array([[1, 1, 1],
       [1, 1, 1]], dtype=int32)
```

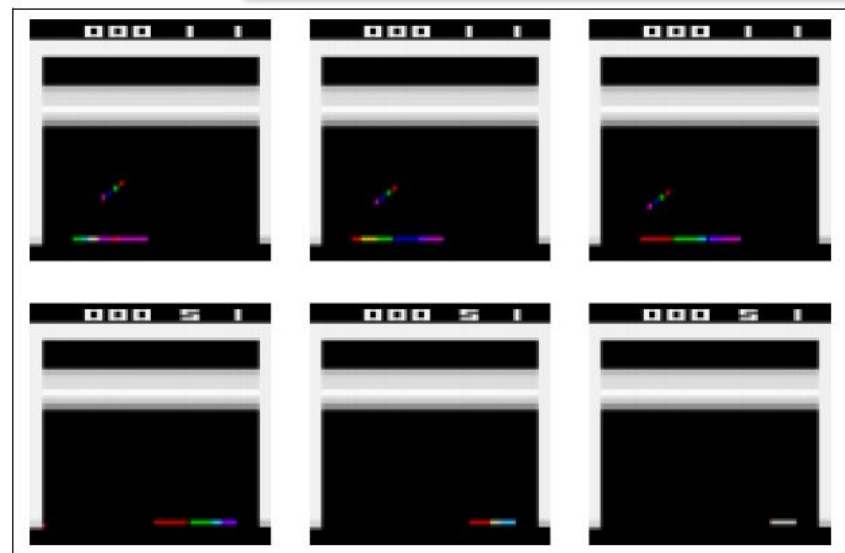


Figure 18-15. Two trajectories containing three consecutive steps each

Creating the Training Loop

(May take hours or days to train depending on hardware)

```
from tf_agents.utils.common import function
```

```
collect_driver.run = function(collect_driver.run)
agent.train = function(agent.train)
```

```
def train_agent(n_iterations):
    time_step = None
    policy_state = agent.collect_policy.get_initial_state(tf_env.batch_size)
    iterator = iter(dataset)
    for iteration in range(n_iterations):
        time_step, policy_state = collect_driver.run(time_step, policy_state)
        trajectories, buffer_info = next(iterator)
        train_loss = agent.train(trajectories)
        print("\r{} loss:{:.5f}".format(
            iteration, train_loss.loss.numpy()), end="")
        if iteration % 1000 == 0:
            log_metrics(train_metrics)
```

```
train_agent(10000000)
```

Overview of Some Popular RL Algorithms

Some Popular RL Algorithms (1)

Actor-Critic algorithms: A family of RL algorithms that combine Policy Gradients with Deep Q-Networks. An Actor-Critic agent contains two neural networks: a policy net and a DQN. The DQN is trained normally, by learning from the agent's experiences. The policy net learns differently (and much faster) than in regular PG: instead of estimating the value of each action by going through multiple episodes, then summing the future discounted rewards for each action, and finally normalizing them, the agent (actor) relies on the action values estimated by the DQN (critic). It's a bit like an athlete (the agent) learning with the help of a coach (the DQN).

Some Popular RL Algorithms (2)

Asynchronous Advantage Actor-Critic (A3C): multiple agents learn in parallel, exploring different copies of the environment.

Advantage Actor-Critic (A2C): All model updates are synchronous, so gradient updates are performed over larger batches, which allows the model to better utilize the power of the GPU.

Soft Actor-Critic (SAC): It learns not only rewards, but also to maximize the entropy of its actions. In other words, it tries to be as unpredictable as possible while still getting as many rewards as possible. This encourages the agent to explore the environment, which speeds up training, and makes it less likely to repeatedly execute the same action when the DQN produces imperfect estimates.

Proximal Policy Optimization (PPO): based on A2C that clips the loss function to avoid excessively large weight updates (which often lead to training instabilities).

Takeaways

- Typical architecture for RL and relevant classes in the TF-Agent API.

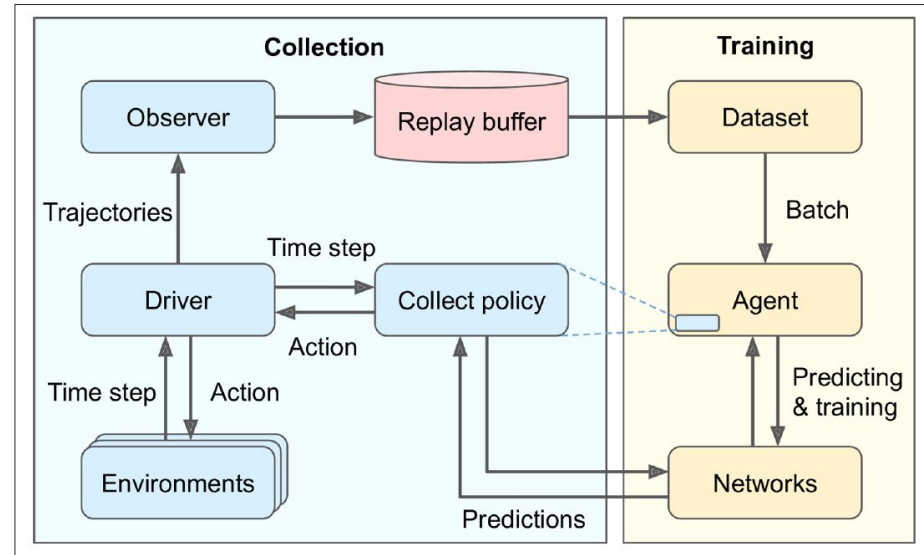


Figure 18-13. A typical TF-Agents training architecture